

url4 topology — endpoint, node, discovery, addressing, transport — open work

Questions, sketches, spec deltas: the part still moving

STATUS	proposed — owner review; one decision left open in §5; reconciled 2026-09-15 with Kevin's Part A §1.4 refresh, PR #19 commit 2a939bff (draft, unmerged) — see §12 and Appendix D
WORK ITEM	OME-1110
DATE	2026-09-04
OWNER	Sergey Bershadsky (execution and telemetry architecture)
GRAMMAR OWNER	Kevin McDonough (URL4 spec, Parts A/B)
SPEC POINTERS	URL4.ai Specification v0.5 DRAFT — Part A §1.4 at PR #19 (2a939bff), Parts A/B on main, Parts C–I drafts on url4-refactor

Read the core document first; this part assumes its vocabulary.

What this part is

The companion to the core document. It carries what is not yet decided: the questions on authentication (§10), the networking sketch (§11), the alignment with Kevin's spec (§12), the peer-to-peer direction (§13), the spec deltas for Kevin (Appendix A), the follow-up work items (Appendix B) and the vocabulary crosswalk (Appendix D). Read the core first; every term used here is defined there.

10. Deferred, and two questions to move forward

Authentication as a node concern (a question, not a decision). Keeping node and endpoint apart gives credentials exactly one owner. That makes a safe place for authentication possible. The spec's inter-node auth (Part H §31, "under active development") sends per-destination encrypted tokens in `ABC-Auth-Token` headers. Part B §3.5 names a `URL4-Auth-Token` with a target type. The SDK defers all of it: `url4 serve` ships no authn or authz and asks for a reverse proxy in front. The Engine already has the shape we want. The App mints a per-run capability token. The Runner only carries it.

The idea to test: **the node, through its evaluator, is the only party that holds, validates and issues credentials.**

Endpoints never see a raw credential. An endpoint gets a node-issued session: a capability scoped to one run (`rid`, request tree, purpose, expiry). It uses that for everything it needs: reading `@` holdings, calling a sibling endpoint, asking the node to fetch an `s3://` source. Outbound, the node forwards the requestor's per-destination tokens and keeps the ones addressed to it (Part H §31.5). Inbound, the node validates before any endpoint runs. A standalone serverless function is its own node and validates for itself. So the rule holds at every size. The draft already fixes the token's bindings: `rid`, a timestamp within 300 seconds, and the destination identity (Part H §31.2).

Questions to answer before this becomes a section:

1. **Session shape.** Is the endpoint-side session the Engine's JWT topic capability, generalised (`sub` = run, `iat` window)? Or the spec's `URL4-Auth-Token`? One token type, or a node-internal one plus the spec's wire one?
2. **Where the identity lives.** `@alice` access control and consent (Part B §5.6.4) need the requestor's identity at the endpoint. Does the session carry it, or does the endpoint ask the node? Part H §33.1 asks the mirror question: per server or per endpoint. "Per node" is our answer.
3. **Proxy mounts.** For `/claudio → url4://beta.example/claudio`, which side validates the requestor? Does beta see our identity or a delegated one (Part H §31.2 encrypts to the destination)?
4. **Token destination.** Part H §31 addresses tokens to a node and matches on authority (§31.5). Is the destination always the node, never an endpoint path?
5. **Agent sessions.** The coordinating node issues the `Session-Id` (Part G §28.5). Is that session bound to the run capability? Does the transcript URL it exposes fall under the same access control?
6. **What an endpoint may do with a session.** Only call back into its own node? Or reach other nodes directly with a node-minted, destination-bound token?

Recommendation to explore first: a node-issued run capability for endpoints (the Engine's pattern), spec §22 tokens between nodes, identity carried in the capability claims. Attribution, consent and audit (Parts E and H) then attach to the same run identity.

A dry-run endpoint on the node (the word is settled; the shape is not). Deferred in the design session. Reopened because three later additions gave it inputs: per-endpoint `accepts` and `emits` (§8), per-endpoint `delivery` and `schemes` (§5, §2), and a node egress that already checks every target's policy before spending (§11). Today the SDK's `Graph.validate()` checks syntax only. The Engine's preflight checks routes on one node.

Kevin has defined it (Part A §1.4.3): evaluate without executing the intent; return the envelope, no result body. That is the meaning proposed here. What is open is how to ask for one, what it costs, and what happens across nodes.

The shape in full, so the questions have something to bite on. The draft already makes an endpoint do most of it: compute the consumer set by static analysis before resolving anything (Part H §29.2.1), be ready to show a source the full call tree (Part H §30.2), and estimate before executing under a hard budget (Part E §24). It is steps 3 and 4 of §11 without step 5. Parse. Resolve mounts. Type-check every edge against `accepts / emits`. Check schemes and delivery modes. Consult policy and budgets. Return the request tree with a verdict per source and an estimated cost. Spend nothing. The answer is the envelope (Part D §17) with `sources[]` and `meta` filled in and no `result`. A refused source carries its error code. So "can this node run my expression?" becomes one call with a structured no.

Questions:

1. **Shape.** `Prefer: dry-run` on the ordinary GET (the RFC 7240 pattern we already use for `respond-async`)? A `dry-run` parameter beside `q=`? A separate path? The header keeps the same address for dry run and run, which suits caching and audit.
2. **Does a dry run spend?** Policy-registry consults and budget reservations are real calls. Is a dry run free by definition, with `meta.total_cost` an estimate from `pricing_version`? Or may it reserve?
3. **Is the result an artifact?** A dry-run id returned as a handle, so a later run can say "execute this" and skip re-evaluating. That ties to idempotency (§7) and to the token binding in §11. The spec's definition returns "only the envelope without a result body", which gives the caller nothing to hold on to.
4. **Federation.** For a remote subtree, does the node forward the dry run to the other node and merge its answer? Or answer only from that node's capabilities document? The first is exact and costs a round trip. The second is instant and approximate. A dry run that quietly stops at the first remote edge is worse than one that says so.
5. **Ownership.** The Engine's preflight admission (OME-880) becomes this endpoint's single-node case. Does the SDK get `url4 dry-run`, calling the same thing?

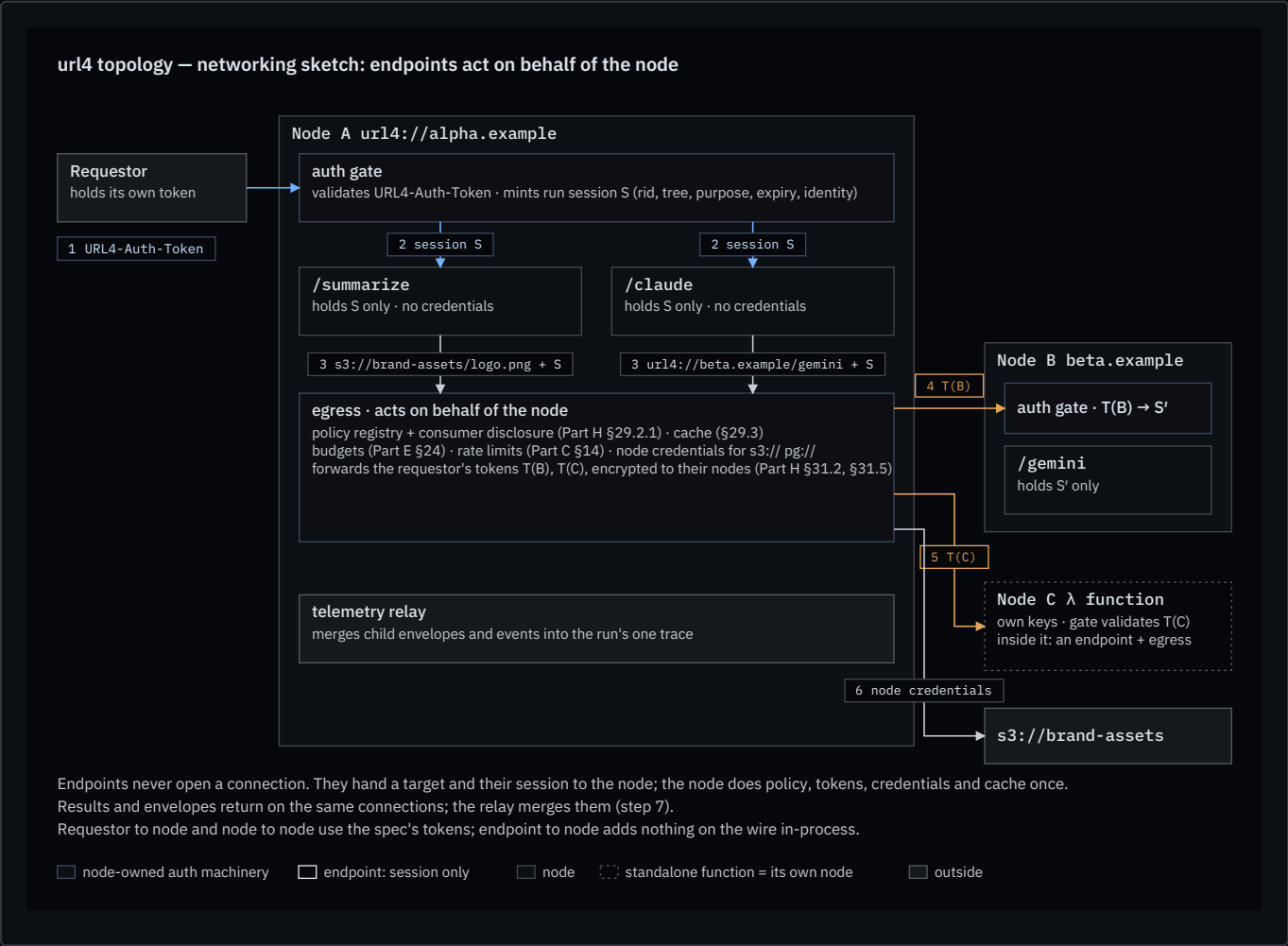
Recommendation to explore first: `Prefer: dry-run` on the same GET, envelope-only answer, cacheable like any GET, forwarded to remote nodes that advertise it, approximated from capabilities where they do not.

Deferred

- **Swarm** as a protocol noun. It would need membership and trust rules that belong to the unwritten governance parts.
- **Endpoint grades** (processor-only vs evaluator). Rejected: every endpoint evaluates.

11. Networking sketch: endpoints act on behalf of the node

A brainstorm, not a decision. It answers one question from §10: **how does an endpoint make an outbound request without holding a credential?**



Three candidate mechanisms

	A · NODE EGRESS	B · DELEGATED TOKEN	C · SIDECAR
Who opens the outbound connection	the node	the endpoint, with a node-minted token	a per-endpoint proxy process
Where credentials live	node only; it forwards the requestor's destination-encrypted tokens and keeps those addressed to it (Part H §31.5)	the requestor's destination-bound token (Part H §31.2), carried by the endpoint	sidecar only
Policy, disclosure, cache, budgets, rate limits	one place: the node's egress (Part H §29.2.1, §16.3, §20, §31)	at mint time on the node; enforcement split	in the sidecar
Serverless endpoint	nothing to configure	needs network egress and key handling per function	platform-dependent
Cost	an extra hop; large payloads through the node, or by reference (§8)	none	one process per endpoint
What it is today	how the SDK already works: an in-process endpoint fetches through the node's <code>IOLayer</code>	the Engine's per-run JWT, generalised	not built

Recommendation to explore first: A for endpoints, B between nodes. An endpoint never talks to the network. It talks to its node. The node talks to other nodes with the spec's tokens. A standalone serverless function is its own node, so the same two rules cover it: the parent node reaches it node-to-node (B), and inside it the function is an endpoint using its own node's egress (A). C is a deployment shape of A, not a third protocol.

The flow the diagram draws

1. The requestor calls Node A with its `URL4-Auth-Token` (Part B §3.5). Node A's auth gate validates it.
2. The gate mints a **run session S**: `rid`, the request tree, purpose, expiry, the requestor's identity. Every endpoint in the run gets S and nothing else. In-process endpoints get it as an object. Command mounts get it in the environment.
3. An endpoint that needs `url4://beta.example/gemini` or `s3://brand-assets/logo.png` hands the target and S to the node's **egress**. It does not open a connection.
4. Egress does the node's work once, in one place. It consults the policy registry and discloses consumers (Part H §29.2.1). It checks budgets and rate limits (Part E §24, Part C §14). It serves from cache when allowed, keyed by requestor identity and consumer set, so a shared cache is not a policy bypass (Part H §29.3). Then it either fetches with the node's own credentials (`s3://`, `pg://`) or forwards the requestor's token **T(B)**, encrypted to Node B, on the sub-request (Part H §31.2, §31.3). The node mints nothing for other nodes. The spec has the originator encrypt every token, so intermediaries cannot read or replace them. What the node mints is internal: the run session S.
5. Node B's gate validates T(B), mints its own session S', and runs `/gemini`. Its result and envelope come back on the same connection.
6. A proxy-mounted standalone function is Node C: same as step 5 with T(C).
7. Node A's telemetry relay merges what came back into the run's one trace (doctrine F1).

What is new on the wire, and what is not

- Requestor to node, and node to node: nothing new. The spec's tokens and `traceparent`.
- Endpoint to node: **nothing on the wire for in-process endpoints**. It is a function call through the node's `IOLayer`, which is what `Url4Node` does today. Out-of-process endpoints (command mounts, containers) need an explicit node endpoint, something like `GET /.node/egress?u=<absolute URI>` with `URL4-Session: S`. Its path and shape are open.

Open questions this sketch adds

- Does egress return bytes to the endpoint, or a reference (§8)? For large sources, a reference keeps the node out of the data path.
- The spec binds T(B) to `rid`, a timestamp (≤ 300 s) and the destination (Part H §31.2). Does the node-internal session S need the same three?
- Delegated credentials only target `abc_node` or `http_source` (Part H §31.4). For `s3://` and `pg://` the node's own credentials are the only path. Should `target_type` widen, or is node-credential-only the rule?
- Can an endpoint ever get direct egress (mechanism B at endpoint level)? Probably only for trusted local mounts, and only by node policy.
- Where does the egress endpoint live for command mounts: a Unix socket, a loopback port, or the node's public origin with S as the credential?

12. Alignment with the spec (2026-09-08 drafts, 2026-09-15 Part A)

The `url4-refactor` branch of `OpenMined/screamingface-design` has draft Parts C–I (cut 2026-04-28 from the v0.4 text, not yet reviewed) and a v0.4 monolith that rennumbers everything after §20. This document was first written from the v0.2 monolith. Every citation now uses the Part numbering. The table shows what the drafts changed and how this document responded.

TOPIC	WE WROTE	THE DRAFT SAYS	RESOLUTION
Capabilities document	schema unwritten	Part G §27.2 defines it: one <code>node</code> , <code>collections</code> , <code>intent_processors</code> , <code>processor_policy</code> , <code>mode_support</code>	§5 rewritten as spec shape + marked additions (<code>path</code> , <code>delivery</code> , <code>schemes</code> , <code>default_processor</code>)
Discovery mechanisms	<code>.well-known</code> vs OPTIONS	<code>.well-known/abc-capabilities</code> SHOULD, <code>Capabilities</code> header MAY (Part G §27.1); no OPTIONS	header added as mechanism C; recommendation now A + C
Sub-paths	<code>/name</code> is an endpoint	sub-paths are collections (Part G §27.4); processors are addressed by <code>id</code>	proposed <code>intent_processors[].path</code> ; vocabulary realigned to the spec's Node/Endpoint (2026-09-08)
Mounts vs delegation	<code>proxy</code> forwards a sub-request	<code>processor=</code> delegation, five forms; <code>abc_delegate</code> gets materialised sources (Part G §27.3)	mounts mapped onto processor types (§4); no new mechanism
Async	<code>Prefer: respond-async</code> , <code>Location</code> handle	<code>delivery=async</code> ; 202 body with <code>poll_url</code> (Part C §9.1, §12.5)	adopted; <code>Prefer</code> and <code>Location</code> become aliases
Wrapper switch	<code>Accept: application/json</code> = envelope	<code>Accept</code> negotiates the result's type; envelope is always JSON (Part C §9.1, Part F §25.2)	dedicated <code>application/url4-envelope+json</code> (§6)
Response ladder	WS → SSE → sync, ours	two edges only (Part C §11.4); degrade-not-fail is a rule (Part C §10.2)	kept as delta, anchored in §10.2
<code>;accept</code> values	MIME types	short aliases only; no image/audio/video rows in the registry (Part F §25.5)	examples fixed; alias rows proposed
Binary results	<code>result.media_type</code>	<code>result.content_type</code> ; data URI in LLM mode, raw for RDS (Part G §26.2.3)	adopted; <code>result.artifact</code> stays a delta
<code>ct_mismatch</code>	fail / convert / pass	four values, default <code>ignore</code> , only with <code>;accept</code> (Part G §26.3)	adopted
Telemetry events, cost event	ours	21 SSE types (Part C §12.5); <code>total_cost</code> scalar, <code>budgets_spent</code> (Part D §17, Part E §24)	kept as additions; roll-up mapped to <code>total_cost</code>
Statelessness	every endpoint keeps nothing	the coordinating node owns an agent session and transcript (Part G §28)	§3: state on the node; endpoints stay stateless
Node-minted tokens	egress mints T(B)	originator encrypts every token; intermediaries forward or hold (Part H §31.2, §31.5)	§11 corrected: node forwards; mints only its internal session
Non-HTTP credentials	node's own	<code>target_type</code> is <code>abc_node</code> <code>http_source</code> only (Part H §31.4)	stated; question for Kevin
Artifacts and proxies	free to forward	flow constraints and redistribution depth (Part H §29.2.2)	cautions added to §4 and §8

TOPIC	WE WROTE	THE DRAFT SAYS	RESOLUTION
Root /	ours	unaddressed; examples use bare <code>abc://node?q=</code> (Part I §41.27); <code>v</code> param outranks the path (Part D §19.4)	kept; tensions listed in §2
Cancel	<code>DELETE</code> on <code>Location</code>	<code>DELETE <poll_url></code> or <code>cancel=<rid></code> (Part C §16.2); <code>cancelled</code> not in the status enum (Part D §17.4)	<code>DELETE <poll_url></code> ; enum gap flagged
Naming	<code>url4://</code> , <code>URL4-*</code>	C-I still <code>abc://</code> , <code>ABC-*</code> ; consolidation is open question 19 (Part I §43)	kept <code>url4</code> ; assumption stated in §1

Confirmed by the drafts, unchanged: the three trust relationships and token bindings (Part H §31), the strict request tree (Part H §29.1), `/name` ensembles on one node (Part I §41.9), a weighted image source (Part I §41.20), consumer disclosure before resolution (Part H §29.2.1), the idempotency wording we dispute (Part C §15.1), and every part of our cancel shape (Part C §16.2).

Kevin's Part A §1.4 refresh (2026-09-15, PR #19 [2a939bfff](#), draft)

A week later Kevin rewrote Part A §1.4 in reply. The flat 26-row table became five sub-tables: **1.4.1 Grammar**, **1.4.2 Network**, **1.4.3 Expression Processing**, **1.4.4 Transport**, **1.4.5 General**. Most of Appendix D went into the spec. Where his text differs from ours, his wins. The table shows what we changed in reply.

TOPIC	WE WROTE	PART A §1.4 NOW SAYS	RESOLUTION HERE
<code>Mount</code>	ours: <code>local / command / proxy</code> ; "not defined" in the spec	§1.4.3, adopted word for word , beside the processor types	§1 and §4 cite the spec; delta 1 retired; the stray "N4" note is delta 15
<code>Evaluator</code> , <code>Evaluation</code>	ours (§1)	§1.4.3, adopted and widened : the evaluator may or may not also be the intent processor	§1 takes the wider reading
<code>Dry run</code>	our open question, called "plan" (§10)	§1.4.3, defined : evaluate without executing; envelope, no result body	§10 narrowed to shape, cost, federation; "plan" retired
Response ladder	ours, WS → SSE → sync	§1.4.4, adopted , marked "SPEC SECTION NEEDS UPDATING"	§6 rewritten as adopted; delta 4 becomes "close the Part C gap"
<code>websocket</code> delivery	our fourth rung, an addition	§1.4.4, a fourth <code>delivery</code> value beside <code>sync / stream / async</code>	§6; settles Part I question 26 for answers
Envelope switch	ours: <code>application/url4-envelope+json</code>	§1.4.4, adopted , with the <code>meta</code> parameter	§6 marked adopted; delta 7 retired
<code>Scheme adapter</code>	ours (§2)	§1.4.3, adopted , "generalizes Part B §3.5"	§2 cites the spec; says "host's credentials" → delta 13
<code>Degradation</code> , <code>Flow constraints</code> , <code>Attribution</code> , <code>Run handle</code> , <code>Collection</code> , <code>Holdings</code>	ours or implied	§1.4.3–§1.4.5, all defined	Appendix D becomes a crosswalk, not a second glossary

TOPIC	WE WROTE	PART A §1.4 NOW SAYS	RESOLUTION HERE
Request Tree	strict tree, never a graph (Part H §29.1)	§1.4.3: "the full set of all sources and targets... an evaluator may expand the tree ". Strictness dropped.	§1 flags it; delta 14 asks if it is on purpose. §13's peer-to-peer question depends on the answer dropped.
"Host"	removed from this document 2026-09-08	§1.4.2 adds Host system (a machine); §1.4.4 and the Scheme adapter row then use "host" to mean node	§1 keeps "host" out; delta 13 proposes s/host/node/ in those four rows
Capabilities path	/.well-known/url4-capabilities	§1.4.5 writes exactly that; Part G still says abc-capabilities	§5: question 19 answered for this path
Self-reference, Identity-reference	two rows in the old §1.4	dropped as rows; folded into Holdings	delta 16 asks for confirmation; Holdings also has Collection 's anchors
Branch	Parts C–I live on url4-refactor	PR #19 targets main , where C–I are stubs, yet cites Part C §10.2, Part G §27.3, Part H §31	delta 17: those anchors do not resolve on the branch the PR merges into

Pinned to [2a939bff](#), a draft with no review (Appendix C). Appendix B has the re-review item.

13. url4 as a network protocol: submit, not call (question)

Raised 2026-09-08. Today a url4 expression is called: one requestor sends a GET to one node and waits. The question: can the same expression be submitted to a network that resolves it over time, the way a torrent is fetched or a transaction is mined? Every requestor is a node. Capable nodes claim the endpoints an expression names. The result finds its way back.

What the spec already gives such a network

- **The expression is the address, and it can be content-addressed.** **urn:sha256:** is the accepted form (Part I §43, question 1). The canonical rendering of an expression hashes to a stable work id.
- **Submit and collect are already separate.** **delivery=async** returns a handle; the result arrives later (Part C §11.3). **resume=<rid>** continues from the last known state (Part C §9.1). Results carry **result_version** and **is_final**, so an answer may improve over time (Part C §12.5).
- **Redundant evaluation has words.** Quorum and triggers say how many of N must succeed before an answer counts (Part C §12). Run the same endpoint on several nodes and quorum becomes agreement.
- **Discovery can be a record.** A node's capabilities document (Part G §27.2) is a self-description. It can live in a distributed table as well as at **/.well-known**.
- **Every requestor is a node.** A local SDK process that evaluates is already a node here (§4).
- **Attribution is an incentive model.** Attribution scores and the still-undefined settlement (Part E, Part I §37) are where a Bitcoin-like reward would attach.

What is genuinely new

- **Claiming work.** Which node evaluates a submitted expression? How do two nodes avoid doing the same work, or do it on purpose?

- **Verifying results without determinism.** A torrent piece is verified by hash. An LLM result cannot be. Candidates: agreement across redundant evaluations (quorum), attested execution (Part I §38), reputation, or accepting unverifiable results for some intents.
- **Consent in an open swarm.** A source may name its allowed consumers and cap redistribution depth (Part H §29.2.2). An anonymous claimer is neither known nor allowed. Either sources opt into open resolution, or the swarm is a closed membership.
- **The context travels.** Submitting an expression means its resolved sources travel to whichever node claims it. That is the opposite of RDS mode's "code to the data" (Part A §1.1).
- **Liveness and economics.** Who guarantees an expression is ever resolved? Who pays? Part I §37 is TBD.

A sketch to argue with. The requestor canonicalises the expression and publishes `urn:sha256:<hash>` with the expression to the network. Nodes whose capabilities cover an endpoint the expression names claim that endpoint. They evaluate it against their own holdings and credentials, as §11 describes for one node. Each returns a signed envelope keyed by the hash. The requestor's `poll_url` becomes a lookup of that key. Quorum decides when the answer is good enough. `is_final` closes it. The grammar does not change. The transport changes from point-to-point to publish-and-claim.

Questions

1. Is this a fourth delivery mode (`delivery=network`), or a different transport under the same three?
2. Which verification candidate is acceptable for which kind of intent?
3. Can consent and flow constraints be expressed for an open set of claimers, or is membership required?
4. Does the requestor's evaluator keep fan-out and reduce, with the network resolving only leaf endpoints? That keeps §11's egress intact and makes the swarm a resolver, not an orchestrator.

Appendix A — proposed spec deltas for Kevin

Each item names the anchor and the change. All are proposals. Re-triaged 2026-09-15 against Part A §1.4 at PR #19 (`2a939bff` , draft).

Landed in PR #19

Kept as a record. Nothing is asked here.

#	WAS	WHERE IT LANDED
1	Part A §1.4 — endpoint kinds. Name the binding kinds <code>local command proxy</code> .	§1.4.3 <code>Mount</code> , word for word. One difference: no rename was needed. Node and Endpoint stayed; §1.4.2 added <code>Node address</code> and <code>Endpoint path</code> .
4	Part C §10.2, §11.4 — response ladder and floor. WebSocket → SSE → sync in one round trip; sync the only MUST; answering below the ask is a reported degradation.	§1.4.4 <code>Response ladder</code> and <code>Degradation</code> , plus <code>websocket</code> as a fourth <code>delivery</code> value. Still open below: Part C itself is unchanged, as Kevin's row notes.
7	Part D §17 — envelope for sync callers. <code>application/url4-envelope+json</code> selects the envelope; <code>meta</code> sets the depth.	§1.4.4 <code>Envelope</code> . It does not state the <code>telemetry</code> block or the <code>meta=summary</code> default; delta 12 still stands.
9	Part B §3.5 — other schemes. Any non-HTTP scheme is a read through a node-mounted adapter, listed in capabilities.	§1.4.3 <code>Scheme adapter</code> , "generalizes Part B §3.5". Says "host's own credentials"; see delta 13.

#	WAS	WHERE IT LANDED
10	Part G §27.2 — capabilities document. Add <code>delivery</code> , <code>schemes</code> ; <code>url4_*</code> naming.	§1.4.5 <code>Capabilities</code> document now lists delivery modes and schemes and writes <code>/.well-known/url4-capabilities</code> . <code>intent_processors[].path</code> , <code>inline_max_bytes</code> and <code>default_processor</code> are still open below.

Still open

- **2. Part B §3.1.1, Part D §19 — root.** `url4://node` $\equiv$ `url4://node/`. `/` is the node's default endpoint at the current version. `/v1` is a version alias. Your own examples do this (Part I §41.27). Also resolve §19.1 (path first) against §19.4 (`v` beats path). Untouched by PR #19.
- **3. Part C §11.2 — bindings.** `delivery=stream` is SSE on the same GET, asked for with `Accept: text/event-stream`. §11.2 names SSE but no media type. Part A §1.4.4 now says `stream` is "SSE events on the same GET", so this is a Part C catch-up.
- **4. Part C §10.2, §11.4 — make Part C match Part A.** §1.4.4 has four delivery modes and the full response ladder. Part C §11 still has three modes; §11.4 has two degradation edges. Your own §1.4.4 row says SPEC SECTION NEEDS UPDATING. Add a `delivery` list per processor to the capabilities document at the same time.
- **5. Part C §15.1 — idempotency.** Replace "the protocol does not guarantee idempotency" with: GET is idempotent per RFC 9110 §9.2.2; results are not guaranteed deterministic. §15.2's `budget_exceeded` row stays as the exception.
- **6. Part C §16 — cancellation.** Take your first option: `DELETE <poll_url>`. Drop `cancel=<rid>`. Terminal state `cancelled`. SSE event `request.cancelled`. Propagates to in-flight children. Partial result when quorum was met. Add `cancelled` to the status enum in Part D §17.4 and the state machine in Part C §13.4.
- **8. Part F §25, Part G §26 — typed payloads.** Add image, audio and video rows with aliases (`png`, `jpeg`, `wav`, `mp4`) to the §25.5 registry. Extend §26.2.3's data-URI rule to results (`result.content` + `result.content_type`). Add `result.artifact` for by-reference results above an endpoint-declared `inline_max_bytes`, subject to §29.2.2 flow constraints. Part E: say how weights and budgets apply to non-text sources. Slot §41.24 is free for a multimodal example.
- **11. Part H §31.4 — credential targets.** Either widen `target_type` beyond `abc_node` | `http_source`, or say that non-HTTP schemes use the node's own credentials only (§2). §1.4.3's `Scheme adapter` assumes the second without saying so.
- **12. Part D §17.2.2, §20 — additions must respect the Hybrid Rule and `propagated`.** Our `telemetry` and `session` blocks are present and `null` at `summary`. Unknown fields pass through unaggregated.

New, raised by PR #19

- **13. Part A §1.4.2, §1.4.4 — host vs node.** `Host system` is fine in §1.4.2 as a deployment word: a machine that runs one or more nodes. But three rows use "host" where the actor is the **node**: §1.4.4 `Delivery mode` ("the negotiated method by which a **host** responds"), §1.4.4 `Response ladder` ("the mechanism by which a **host** responds with the richest delivery mode"), and §1.4.3 `Scheme adapter` ("using the **host's** own credentials"). The node negotiates delivery, picks a rung and holds scheme credentials. A host system does none of that. On a machine running two nodes the sentence has no referent. Proposal: `s/host/node/` in those rows; keep `Host system` in §1.4.2 only.
- **14. Part A §1.4.3 — Request Tree lost its strictness.** The new row reads: "the full set of all **sources** and **targets** defined in an **expression**. An **evaluator** may expand the tree during **source resolution** and sub-expression **evaluation**." Part H §29.1 says the request tree is strictly a tree, never a graph. This document relies on that in §3, §7 and §13. Is the change on purpose — may a node merge two branches onto one sub-expression? Or does "expand" only mean the tree is found lazily? The answer decides whether §13's publish-and-claim model is a transport change or a model change.
- **15. Part A §1.4.3 — a doctrine reference leaked into the spec.** The `Mount` row marks `command` as "(subprocess, N4)". N4 is an item in our internal doctrine skill (`.claude/skills/url4-engine/SKILL.md`), not a spec anchor. A reader of Part A cannot resolve it. Drop it, or replace it with Part H §36, the rule that governs command mounts.

- **16. Part A §1.4.1 — Holdings anchors, and two dropped rows.** Holdings cites Part B §5.3, Part G §27.4. Those are Collection's anchors. Holdings and the @ token are Part B §5.6. Also, Self-reference and Identity-reference were rows in the old §1.4 and are gone, folded into Holdings it seems. Please confirm. @alice carries an access-control and consent rule (Part B §5.6.4) that the one Holdings line does not state.
- **17. Branch and anchors.** PR #19 targets main, where Parts C–I are "Not yet written" stubs. Its new rows still cite Part C §10.2, Part C §11, Part C §12.5, Part G §27.3 and Part H §31. Those resolve only on url4-refactor. Either the drafts merge first, or Part A §1.4 ships with references a reader of main cannot follow.
- **18. Part A §1.4 — typos and markup.** Inter-host tone → Inter-host token (and, per delta 13, inter-node token). Request identifier x2 → identifier. Extention → extension. Unclosed bold in "nested **Expression**" (§1.4.1 Source) and "one or more **nodes**" (§1.4.2 Host system). "one **ore** more" (§1.4.3 Resolution). "without executing **then** intent" (§1.4.3 Dry run). "advertising **a its** collections" (§1.4.5 Capabilities document). A stray . before "Generalizes" in Scheme adapter.

Appendix B — follow-up work items

To file after owner review, one per landing.

LANDING	ITEM
screamingface-engine	Node surface phase 1: discovery per §5, GET /?q= with SSE, DELETE on the run handle
screamingface-engine	Model endpoints reachable over HTTP (phase 2); standalone model functions, proxy-mounted (phase 3)
screamingface-engine	Rename the URL4-Capability JWT header to avoid the "capabilities" collision
url4-sdk	Delivery negotiation in Url4Node.asgi(): Upgrade / Accept handling, application/url4-envelope+json, SSE body, delivery=async with poll_url, optional WebSocket answer; evaluator-side single-request response ladder in HttpIOLayer
url4-sdk	Capabilities document and/or OPTIONS responder, after the §5 decision; advertise delivery per endpoint
url4-sdk	url4 serve default path /; Client default path /; /v1 alias
url4-sdk	proxy mount kind in url4.toml
url4-sdk	Scheme adapters (s3://, pg://, sqlite://, ...) as IOLayer adapters registered by scheme; [schemes] in url4.toml; schemes in capabilities
url4-sdk	Typed payloads: bytes + media type through IOLayer / FetchRequest; ;accept / ct_mismatch enforcement; result.artifact by-reference fetch
screamingface-engine	Non-text processors over aigateway (image, speech); artifact store as the by-reference path; accepts / emits in capabilities
url4-sdk	Url4Node → node naming retrofit with a deprecation alias
repo	Doctrine skill synced in this unit (T1, N1, F2, F4, term table)
repo	Re-review this document when PR #19 leaves draft. Every "adopted" claim is pinned to 2a939bff; the PR has had no review
Kevin	Review Appendix A. Deltas 2, 3, 4, 5, 6, 8, 11, 12 still open; 13–18 raised by PR #19 itself

Appendix D — Vocabulary crosswalk

This was a glossary. Now Part A §1.4 defines most of these terms, and a second glossary would only drift. So this is a crosswalk: for each term, where it lives in Part A §1.4 at `2a939bff`, and what is still owed. Read the definitions in the spec.

Status key. adopted — in Part A §1.4, saying what we meant · changed — adopted, but the meaning moved; see the delta · spec's own — the spec's word before this document existed · ours — not in the spec; this document is the only definition.

Adopted from this document

TERM	PART A §1.4	STATUS	STILL OWED
Mount	§1.4.3	adopted	The row marks <code>command</code> with "N4", our doctrine numbering (delta 15).
Evaluator, Evaluation	§1.4.3	adopted, widened	The evaluator "may or may not" also be the intent processor. Wider than §1 had it; we take the wider reading.
Dry run	§1.4.3	adopted	The operation is defined. How to ask for one, what it costs and how it federates are open (§10).
Response ladder	§1.4.4	adopted	Part C §11.4 still lists two edges. Kevin's row says SPEC SECTION NEEDS UPDATING (delta 4).
Degradation	§1.4.4	adopted	—
Scheme adapter	§1.4.3	adopted	Says "the host's own credentials". It is the node's (delta 13).
Envelope	§1.4.4	adopted	Has the <code>Accept: application/url4-envelope+json</code> switch and <code>meta</code> . The <code>telemetry</code> block and the <code>meta=summary</code> default are still deltas (12).
Run handle, <code>poll_url</code>	§1.4.4	adopted	—
Flow constraints	§1.4.5	adopted	—
Attribution	§1.4.3	adopted	—
Delivery mode	§1.4.4	adopted	<code>websocket</code> is the fourth value; <code>sync</code> the only MUST.
Request tree	§1.4.3	changed	Kevin writes Request Tree; we keep sentence case, as his other terms do. Strictness is gone: "an evaluator may expand the tree". Part H §29.1 says strictly a tree. §3, §7 and §13 depend on which is true (delta 14).
Inter-node token	§1.4.4	changed	Appears as "Inter-host tone ": a typo, and "host" for "node" (deltas 13, 18).

The spec's own, before this document

TERM	PART A §1.4	NOTE
Expression, Source, Intent	§1.4.1	<code>Intent</code> is newly spelled out: a prompt, a code pointer, a URI, or an expression that must resolve to one of the first two.
Collection, Holdings	§1.4.1	<code>Holdings</code> is a new row. It folds in the old <code>Self-reference</code> / <code>Identity-reference</code> . Its anchors are <code>Collection</code> 's (delta 16).

TERM	PART A §1.4	NOTE
Broadcast, Expansion	§1.4.1	Unchanged.
Node, Endpoint	§1.4.2	<code>Endpoint</code> is now an interface, named by an endpoint path . See below.
Requestor, Target	§1.4.3	<code>Target</code> is now node address + endpoint path.
Intent processor	§1.4.3	Unchanged in substance.
Resolution, Execution	§1.4.3	Unchanged.
Terminal state, Quorum, Trigger	§1.4.3	<code>Trigger</code> now says intermediate result.
Agent session	§1.4.3	The basis for the statelessness exception in §3.
Policy registry, Data owner, Purpose, Jurisdiction, Settlement, Attestation	§1.4.5	Governance words. Unchanged.
URL4-aware, Non-URL4 source	§1.4.5	Unchanged.
Capabilities document	§1.4.5	Now written <code>/well-known/url4-capabilities</code> ; names the <code>Capabilities</code> header (§5).
Idempotent	Part C §15	Not in §1.4. Wording still disputed (delta 5).

New in PR #19, adopted here

TERM	PART A §1.4	HOW THIS DOCUMENT USES IT
Node address	§1.4.2	§2: scheme + RFC 3986 authority.
Endpoint path	§1.4.2	§1, §2, §3: <code>/claude</code> is the path; the endpoint is the interface it names.
Host system	§1.4.2	Not used (§1, delta 13).

Still only ours

Nothing in Part A §1.4 defines these. Each is a live proposal with a delta behind it.

TERM	DEFINITION	WHERE
Typed payload	Every edge carries a value named by its media type. Text is the default. Sources declare by <code>Content-Type</code> , results by <code>result.content_type</code> .	§8, delta 8
Artifact	A large result returned as an <code>https://</code> URL the next endpoint fetches as data, above an endpoint-declared <code>inline_max_bytes</code> . Inherits flow constraints.	§8, delta 8
Run session	A node-issued capability scoped to one run (<code>rid</code> , tree, purpose, expiry, identity). Endpoints carry it instead of credentials.	§10, §11
Egress	The node component that makes every outbound request for its endpoints: policy, disclosure, cache, budgets, credentials, token forwarding.	§11
Telemetry signals	Logs, spans (tokens live here), and <code>cost.usage</code> events (money lives here). In-band in every mode. OTLP is the durable copy.	§7, delta 7

TERM	DEFINITION	WHERE
OPTIONS discovery	One answer per endpoint, mechanism B of §5. Not in the spec. PR #19 did not add it.	§5